

Assignment 5: Build and Evaluate Classification Models

Juan Maldonado Franco

DDS-8555 Predictive Analysis

Mohamed Nabeel

```
In [1]: from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

ROOT = Path.cwd()
for parent in [ROOT, *ROOT.parents]:
    if (parent / "DDS-8555 - Predictive Analysis").exists():
        COURSE = parent / "DDS-8555 - Predictive Analysis"
        break
else:
    COURSE = ROOT.parents[1]
DATA = COURSE / "data"
KAGGLE = DATA / "kaggle"
SUBMISSIONS = DATA / "submissions"
RANDOM_STATE = 42
pd.set_option("display.max_columns", 80)
```

Conceptual Question 1

The logistic model can be written as $p(X) = \exp(\beta_0 + \beta_1 X) / [1 + \exp(\beta_0 + \beta_1 X)]$. Dividing $p(X)$ by $1 - p(X)$ cancels the denominator and leaves $\exp(\beta_0 + \beta_1 X)$. Taking the natural log gives $\log[p(X)/(1 - p(X))] = \beta_0 + \beta_1 X$. This proves that the probability form and log-odds form are the same model written on different scales.

Applied Question 13: Weekly Classification

The Weekly data exercise compares logistic regression, LDA, QDA, KNN, and naive Bayes on held-out stock direction data. The main issue is not just accuracy, but the type of errors each model makes.

```
In [2]: from ISLP import load_data
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDisc
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
```

```

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.pipeline import Pipeline

weekly = load_data("Weekly")
display(weekly.describe())
train = weekly["Year"] <= 2008
test = ~train
X_train = weekly.loc[train, ["Lag2"]]
X_test = weekly.loc[test, ["Lag2"]]
y_train = weekly.loc[train, "Direction"]
y_test = weekly.loc[test, "Direction"]
models = {
    "Logistic": LogisticRegression(max_iter=1000),
    "LDA": LinearDiscriminantAnalysis(),
    "QDA": QuadraticDiscriminantAnalysis(),
    "KNN-1": Pipeline([("scale", StandardScaler()), ("model", KNeighborsClassifier(
    "Naive Bayes": GaussianNB(),
}
rows = []
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    rows.append({"model": name, "accuracy": accuracy_score(y_test, pred), "confusion_matrix": confusion_matrix(y_test, pred)})
pd.DataFrame(rows)

```

	Year	Lag1	Lag2	Lag3	Lag4	Lag5	Vc
count	1089.000000	1089.000000	1089.000000	1089.000000	1089.000000	1089.000000	1089.000000
mean	2000.048669	0.150585	0.151079	0.147205	0.145818	0.139893	1.511613
std	6.033182	2.357013	2.357254	2.360502	2.360279	2.361285	1.681418
min	1990.000000	-18.195000	-18.195000	-18.195000	-18.195000	-18.195000	0.000000
25%	1995.000000	-1.154000	-1.154000	-1.158000	-1.158000	-1.166000	0.333333
50%	2000.000000	0.241000	0.241000	0.241000	0.238000	0.234000	1.000000
75%	2005.000000	1.405000	1.409000	1.409000	1.409000	1.405000	2.000000
max	2010.000000	12.026000	12.026000	12.026000	12.026000	12.026000	9.333333

Out[2]:

	model	accuracy	confusion_matrix
0	Logistic	0.625000	[[9, 34], [5, 56]]
1	LDA	0.625000	[[9, 34], [5, 56]]
2	QDA	0.586538	[[0, 43], [0, 61]]
3	KNN-1	0.490385	[[22, 21], [32, 29]]
4	Naive Bayes	0.586538	[[0, 43], [0, 61]]

Kaggle Obesity Classification

The competition required multinomial logistic regression, LDA or QDA, naive Bayes, and SVM submissions. These models represent different assumptions about decision boundaries and feature distributions.

```
In [3]: from sklearn.compose import ColumnTransformer
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import LinearSVC

obesity = pd.read_csv(KAGGLE / "playground-series-s4e2" / "train.csv")
X = obesity.drop(columns=["NObeyesdad"])
y = obesity["NObeyesdad"]
cat = X.select_dtypes(include="object").columns.tolist()
num = [c for c in X.columns if c not in cat + ["id"]]
pre_dense = ColumnTransformer([("cat", OneHotEncoder(handle_unknown="ignore", sparse=False)),
                                ("num", StandardScaler())],
                              remainder="passthrough",
                              n_jobs=-1)
X_train, X_valid, y_train, y_valid = train_test_split(X, y, test_size=.2, stratify=y)
models = {
    "Multinomial logistic": LogisticRegression(max_iter=2000, C=1.0),
    "LDA": LinearDiscriminantAnalysis(),
    "Naive Bayes": GaussianNB(),
    "Linear SVM": LinearSVC(C=.5, random_state=RANDOM_STATE),
}
rows = []
for name, clf in models.items():
    model = Pipeline([("pre", pre_dense), ("model", clf)])
    model.fit(X_train, y_train)
    pred = model.predict(X_valid)
    rows.append({"model": name, "validation_accuracy": accuracy_score(y_valid, pred)})
pd.DataFrame(rows)
```

Out[3]:

	model	validation_accuracy
0	Multinomial logistic	0.868738
1	LDA	0.822977
2	Naive Bayes	0.585983
3	Linear SVM	0.748555

```
In [4]: from pathlib import Path
status_path = SUBMISSIONS / "kaggle_submission_status_playground-series-s4e2.txt"
text = status_path.read_text(encoding="utf-8", errors="ignore")
print("\n".join([line for line in text.splitlines() if "A5_" in line or "fileName"])))
```

Interpretation

The SVM and multinomial logistic models were stronger than naive Bayes on the Obesity data because the predictors are not conditionally independent in a realistic health-behavior data set. LDA was competitive but more assumption-bound because it relies on class-conditional normality and shared covariance structure. The Kaggle evidence confirms all four required submissions were completed.

References

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273-297.
<https://doi.org/10.1007/BF00994018>

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An introduction to statistical learning: With applications in Python*. Springer. <https://doi.org/10.1007/978-3-031-38747-0>

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427-437.
<https://doi.org/10.1016/j.ipm.2009.03.002>